



Templates

Objectives

- In this chapter, you will
 - Learn about templates
 - Explore how to construct function templates and class templates

Templates

- Template: a single code body for a set of related functions (function template) and related classes (class template)
- Syntax:

```
template <class Type>  
declaration;
```

- `Type` is the data type
- Declaration is either a function declaration or a class declaration

Templates (cont'd.)

- `class` in the heading refers to any user-defined type or built-in type
- `Type`: a formal parameter to the template
- Just as variables are parameters to functions, data types are parameters to templates

Function Templates

- Syntax of the function template:

```
template <class Type>  
function definition;
```

- `Type` is a formal parameter of the template used to:
 - Specify type of parameters to the function
 - Specify return type of the function
 - Declare variables within the function

Function Templates-Example

```
#include<iostream>
using namespace std;

template <class T>
T maxT(T a, T b)
{
    if (a>b)
        return a;
    else
        return b;
}

int main()
{
    cout<<maxT(10,20)<<endl;
    cout<<maxT('A','B')<<endl;
    cout<<maxT(1.2,3.9)<<endl;

    return 0;
}
```

OUTPUT

20
B
3.9

Class Templates

- Class template: a single code segment for a set of related classes
 - Called parameterized types
- Syntax:

```
template <class Type>
class declaration
```
- A template instantiation can be created with either a built-in or user-defined type
- The function members of a class template are considered to be function templates

Class Templates-Example

```
#include<iostream>
using namespace std;

template <class T>
class Point
{
public:
    Point() {} // default constructor
    Point(T xVal,T yVal) { setXY(xVal,yVal); }
    void setXY(T xVal, T yVal);
    void print()
    {
        cout<<'('<<x<<','<<y<<')'<<endl;
    }
private:
    T x;
    T y;
};

template <class T>
void Point<T>::setXY(T xVal, T yVal) //setter or mutator
{
    x= (xVal>=0)?xVal:0;
    y= (yVal>=0)?yVal:0;
}
```

```
int main()
{
    Point <int>P1(10,20);
    P1.print();

    Point <double>P2(3.5,6.6);
    P2.print();
}
```

OUTPUT

```
(10,20)
(3.5,6.6)
```